

1단계 — 함수 호출 구조 이해

학습목표

컴퓨터는 “함수 하나 호출할 때마다 작업공간을 새로 만든다”는 것을 이해한다.

그 작업공간이 “스택 프레임(**stack frame**)”인 것을 확인한다.

개념

내용	익혀야할 능력
함수 호출 시 새로운 스택 프레임 생성	재귀 이해의 핵심
매개변수 복사	값 전달 이해
지역변수 생성/소멸	메모리 감각
return 후 스택 제거	함수 종료 이해
함수마다 독립 변수 존재	재귀 혼동 방지

함수 호출 시 새로운 스택 프레임 생성 (재귀 이해의 핵심)

```
void count(int n)
{
    if (n == 0)
        return;

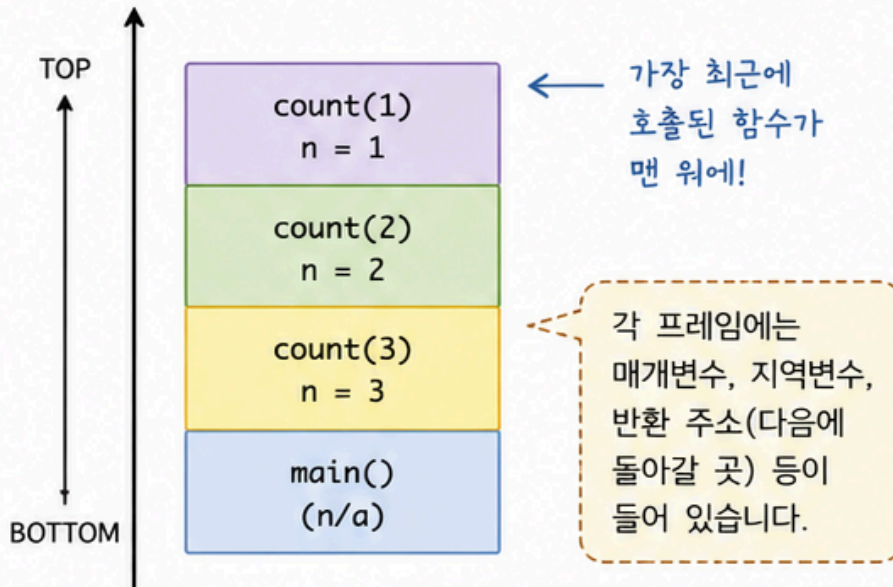
    count(n - 1);
}

int main()
{
    count(3);
    return 0;
}
```

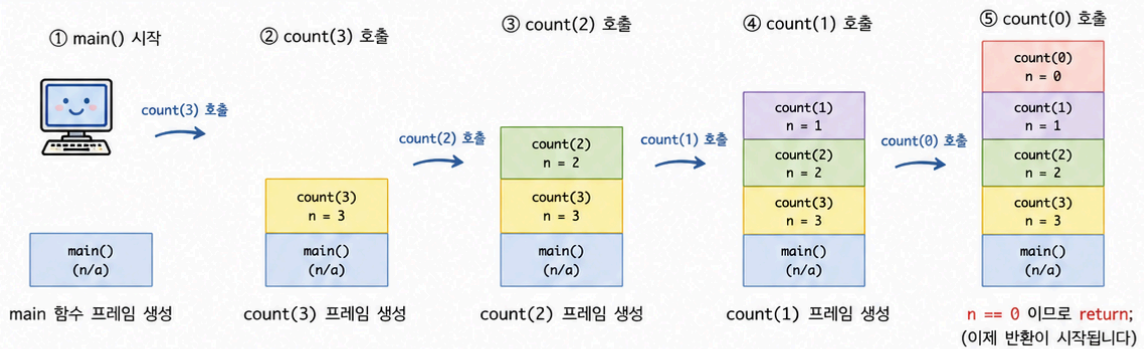
★ 핵심 개념

- 함수가 호출될 때마다 컴퓨터는 그 함수만을 위한 새로운 작업 공간(**스택 프레임**)을 스택 위에 만들어 올립니다.
- 함수가 끝나면 가장 위에 있는 프레임부터 **제거(POP)**됩니다.
- 이 구조가 ‘스택(Stack)’ (후입선출: LIFO)입니다.

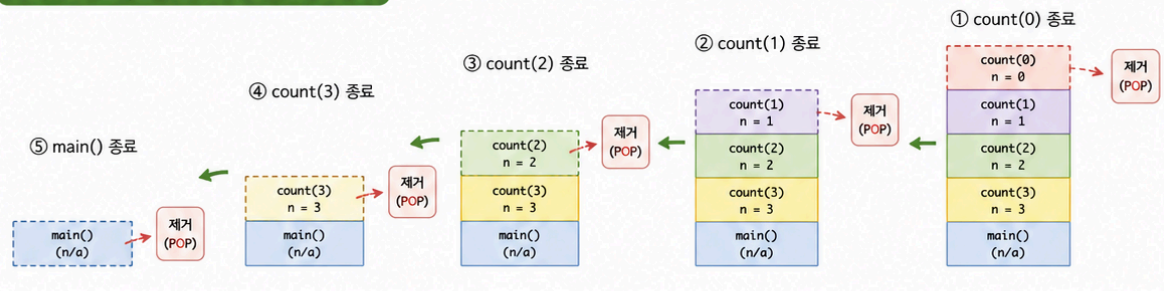
스택의 모습 (위가 스택의 TOP)



1. 호출 과정 (스택 프레임이 쌓이는 과정)



2. 반환 과정 (스택 프레임이 제거되는 과정)



정리: 함수 호출 = 새로운 스택 프레임 생성(푸시) / 함수 종료 또는 `return` = 스택 프레임 제거(팝)
재귀는 이 과정이 반복되면서 스택이 깊어졌다가, 조건을 만족하면 다시 차례로 돌아오며 비워지는 구조입니다.

실습

터미널에서 경로생성 **funk** 후, 그 경로에서 **vsc** 열어서 해당 경로내에 **funk01.c** 파일 만들기

```
cd /c
mkdir c_project
cd c_project
mkdir funk
code .
```

VSC 열리면 파일 **funk01.c** 생성한 후 아래 코드 작성한다.

tip.

int main(){ 형식으로 설정하기

VSC에서 Ctrl + Shift + P 단축키
Open User Settings (JSON) 검색
"C_Cpp.clang_format_style": "{ BasedOnStyle: Google, BreakBeforeBraces: Attach }" 추가

첫 예제

```
#include <stdio.h>
```

```
void test(int x)
{
    int y = x + 1;
    printf("x=%d y=%d\n", x, y);
}
```

```
int main()
{
    test(3);
    test(5);

    return 0;
}
```

컴파일 후, 실행해 본다.

```
funk 경로 위치에서 gcc funk01.c -o funk01
funk 경로 위치에서 ./funk01
```

결과

```
x=3 y=4
x=5 y=6
```

확인!

- x,y는 어디 저장?
- 두 번째 호출 때 첫 번째 y 살아있나?
- 함수 끝나면 변수는?

작성!

- 스택(Stack) 메모리
- 첫 번째 호출 끝나는 순간 사라짐
- 지역변수(local variable)는 사라짐

더 나아가기!

메모리 구조는 컴퓨터에서 이렇게 나뉨

코드 영역
전역변수 영역 ← g
static 변수
Heap ← malloc
Stack ← 함수 지역변수

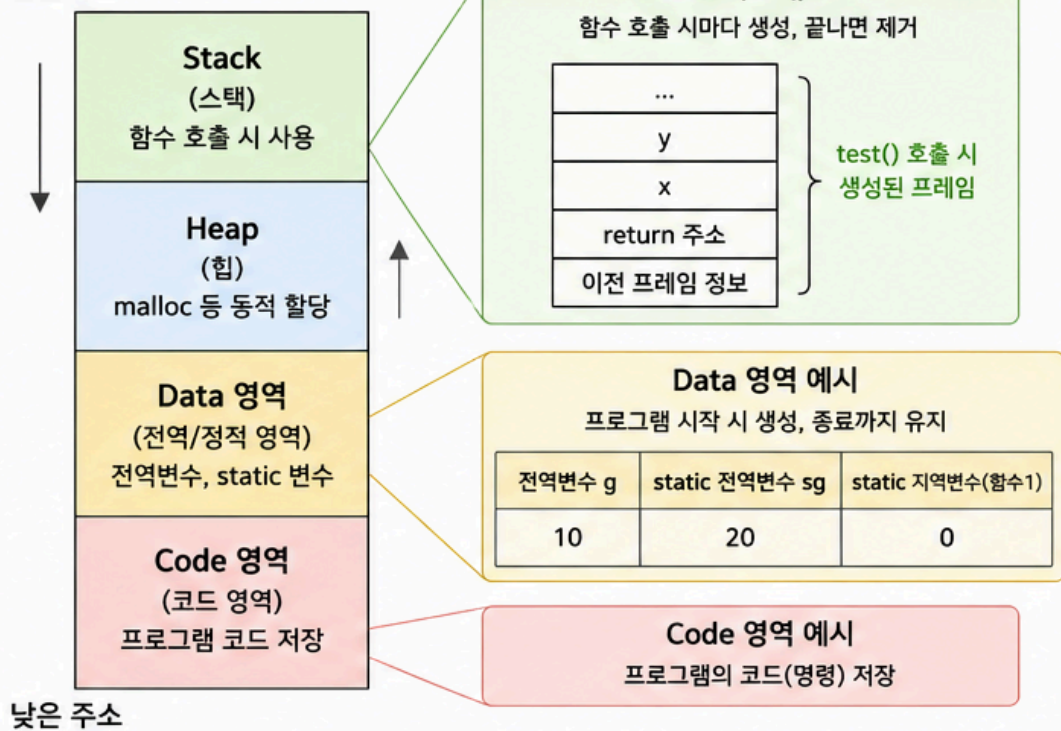
변수 종류	저장 위치	생존 시간
지역변수	Stack	함수 끝날 때 제거
매개변수	Stack	함수 끝날 때 제거
전역변수	Data 영역	프로그램 종료까지
static 변수	Data 영역	프로그램 종료까지

2단계 — 스택 메모리 시각화

개념

1. 메모리 영역 전체 그림

높은 주소

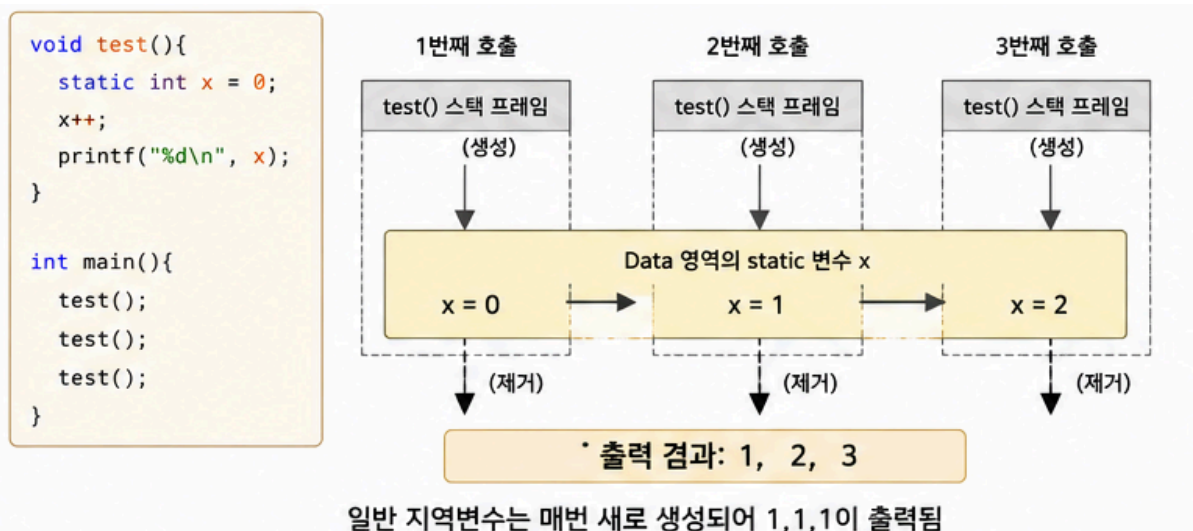


2. 변수 종류별 정리

종류	선언 위치	저장 위치	생존 시간	접근 범위	특징
지역변수 (auto)	함수 안	Stack	함수 끝날 때까지	선언된 함수 내부	호출될 때마다 생성/제거
매개변수	함수 안 (매개변수)	Stack	함수 끝날 때까지	선언된 함수 내부	함수 호출 시 값이 복사됨
전역변수 (global)	함수 밖	Data 영역	프로그램 종료까지	모든 함수, 모든 파일	어디서든 접근 가능 (위험할 수 있음)
static 지역변수	함수 안 (static)	Data 영역	프로그램 종료까지	선언된 함수 내부	한 번만 생성, 값 유지됨
static 전역변수	함수 밖 (static)	Data 영역	프로그램 종료까지	현재 파일 내부만	다른 파일에서 접근 불가

static 두가지 의미

(A) 함수 안 **static** 변수 → “값 유지” (생존 시간 연장)



(B) 함수 밖 **static** 변수 → “접근 범위 제한” (파일 내부만 접근)

파일1.c

```
static int g = 100; // static 전역변수
void test(){
    printf("%d\n", g);
}
```

* 특징

파일1.c 안에서는 g 사용 가능
파일2.c 에서는 g 사용 불가

파일2.c

```
extern void test();
int main(){
    // printf("%d\n", g); // ✗ 오류! 접근 불가
    test();
}
```

메모리(Data 영역)

g = 100

(프로그램 종료까지 유지)

예:

```
main()
└─ test(3)
    x=3
    y=4
```

호출 끝:

main()

스택 제거 라는 것 알아야 함.

3단계 — 재귀 함수 기초

중요한 건:

“함수가 자기 자신을 호출한다”

가 아니라 다음 의미임.

“함수 호출이 계속 스택에 쌓인다”

예제

```
int fact(int n)
{
    if(n == 1)
        return 1;

    return n * fact(n - 1);
}
```

여기서 헛갈리는 부분:

```
return n * fact(n - 1);
```

이걸:

“한 번에 계산된다”

고 생각함.

실제로는:

- 호출 계속 쌓이고
- 맨 아래서부터 **return** 되며 계산됨.

4단계 — 호출 스택 추적

이게 핵심.

예:

```
fact(4)
```

직접 써보기

호출 과정

```
fact(4)
→ 4 * fact(3)
```

```
fact(3)
→ 3 * fact(2)
```

```
fact(2)
→ 2 * fact(1)
```

fact(1)
→ 1

반환 과정

fact(2)
→ $2 * 1 = 2$

fact(3)
→ $3 * 2 = 6$

fact(4)
→ $4 * 6 = 24$

여기서:

- 호출 단계
- 반환 단계

를 분리해서 설명해보기

대부분 “반환 과정” 못봄

5단계 — 재귀 심화

- 스택 깊이
- 메모리 사용
- 종료조건
- 무한재귀
- **stack overflow**

를 연결할 수 있어야 함

반드시 확인할 코드

```
void boom()  
{  
    boom();  
}
```

실행시키고:

- 프로그램 죽는 거

- stack overflow

확인해 보기.

“재귀는 위험할 수도 있다”

감각 가져야 함

6단계 — 디버거 추적

매우 중요!!!

VS Code 디버거로:

- Step Into
- Call Stack 창

직접 확인하기

“아 함수가 진짜 쌓이는구나”

체감 가능

추천 실습 흐름

단계	실습
1	일반 함수 호출
2	지역변수 주소 출력
3	스택 그림 그리기
4	factorial 재귀
5	fibonacci 재귀
6	무한재귀
7	디버거 호출스택 추적

디버거는 “실행”이 목적이 아니라:

- 함수 호출 순간
- 변수 생성 시점
- 값 변화
- 함수 끝나면 변수 사라지는 것
- **static**이면 유지되는 것

을 눈으로 확인하는 도구

피보나치 재귀는 왜 좋냐

```
int fib(int n)
{
    if(n <= 1)
        return n;

    return fib(n-1) + fib(n-2);
}
```

이건:

- 호출 폭발
- 중복 계산
- 스택 증가

가 눈에 보임

여기서:

- 재귀 단점
- 시간복잡도
- DP 필요성

까지 연결 해 이해하기

마무리 질문

못 풀면 이해 못한 것

```
void test(int n)
{
    printf("%d\n", n);

    if(n > 0)
        test(n - 1);

    printf("%d\n", n);
}
```

`test(3)` 출력 결과 예측.

이 문제는:

- 호출 순서
- 반환 순서
- 스택 흐름

전부 이해해야 풀 수 있음